

Ros2 Day3 hw2 보고서

휴머노이드 수습단원 이주원

목차

1. 프로젝트 개요
2. 코드 구조
3. 파일 구조
4. 실행결과

(4. 실행결과 부터 보는 것을 추천 합니다.)

1. 프로젝트 개요

이 프로젝트는 신호등과 교차로를 흉내 낸 시뮬레이터다. 신호가 정해진 시간에 따라 바뀌고, 차량이 일정 간격으로 생성되어 신호에 맞춰 움직이거나 정지선 앞에 멈춘다. 그 모습을 Qt로 만든 창에 실시간으로 보여준다.

노드는 세 개로 나누었다. 신호등 노드는 현재 신호를 관리하고, 차량 노드는 신호를 받아 차량의 위치를 계산하며, Qt 쪽은 두 노드가 보내는 값을 받아 화면에 그리기만 한다. 계산은 전부 노드 쪽에서 하고 Qt는 로직을 가지지 않는다.

신호등 노드는 빨강, 초록, 노랑 순으로 상태를 바꾸며 현재 상태를 문자열로 발행한다. 차량 노드는 그 신호를 구독해서 초록이면 정상 주행, 노랑이면 감속, 빨강이면 정지선 앞에 멈추도록 위치를 갱신하고, 차량들의 위치를 배열에 담아 발행한다. 앞차와의 간격도 유지하도록 해서 차들이 겹치지 않게 했다.

각 신호의 지속 시간과 차량의 속도, 정지선 위치, 생성 주기 같은 값들은 소스 코드에 넣지 않고 yaml 파일에서 관리한다. 이렇게 하면 코드를 다시 빌드하지 않고 값만 바꿔서 동작을 조절할 수 있다. 세 노드는 launch 파일 하나로 한 번에 실행한다.

2. 코드 구조

```
TrafficLightNode::TrafficLightNode() : Node("traffic_light_node")
```

```
{  this->declare_parameter("r_time",5.0);
    this->declare_parameter("g_time",5.0);
    this->declare_parameter("y_time",5.0);
```

```
    r_time_ = this->get_parameter("r_time").as_double();
```

```
    ...
```

```
    publisher_ = this->create_publisher<std_msgs::msg::String>("traffic_light", 10);
```

```
    timer_ = this->create_wall_timer(100ms, [this]() { timer_callback(); }); }
```

파라미터를 선언하고 값을 읽어온 뒤 발행자와 타이머를 등록하는 부분. 선언할 때 적는 값은 yaml이 없을 때 쓰이는 기본값이고, launch로 실행하면 yaml의 값이 들어온다.

```
void TrafficLightNode::timer_callback()
```

```
{
```

```

elapsed_ += 0.1;
if(state_ == "RED" && elapsed_ >= r_time_ )
{
    state_ = "GREEN";
    elapsed_ = 0;
    ...
}
else if(state_ == "GREEN" && elapsed_ >= g_time_ )
...
std_msgs::msg::String msg;
msg.data = state_;
publisher_>publish(msg); }

```

0.1초마다 경과 시간을 더하고, 정해진 시간을 넘으면 다음 상태로 넘어가는 부분. 상태가 바뀌지 않아도 현재 상태는 매번 발행해서 나중에 켜진 노드도 바로 현재 신호를 알 수 있게 했다.

```

subscriber_ = this->create_subscription<std_msgs::msg::String>(
    "traffic_light", 10, [this](const std_msgs::msg::String::SharedPtr msg) {
light_callback(msg); }
);
publisher_ = this->create_publisher<std_msgs::msg::Float32MultiArray>("vehicle_pos", 10);

```

```

auto period = std::chrono::milliseconds((int)(1000.0 / update_hz_));
timer_ = this->create_wall_timer(period, [this]() { timer_callback(); });

```

차량 노드가 신호를 구독하고 위치를 발행할 준비를 하는 부분. 갱신 주기도 파라미터로 받아서 계산한다.

```

void VehicleNode::light_callback(const std_msgs::msg::String::SharedPtr msg)
{ light_ = msg->data; }

```

신호가 들어오면 값만 저장해두는 부분. 실제 계산은 타이머 쪽에서 한다.

```

spawn_elapsed_ += dt;
if (spawn_elapsed_ >= spawn_period_)
{ vehicles_.push_back(0.0);
    spawn_elapsed_ = 0.0; }

```

정해진 주기마다 차량을 하나씩 만드는 부분. 차량은 위치 값 하나로만 표현해서 벡터에 넣

어두었다.

```
double v = speed_;
if (light_ == "YELLOW")
{
    v = speed_ * 0.5;
}
double next = vehicles_[i] + v * dt;
```

```
if (light_ == "RED" && vehicles_[i] <= stop_line_)
{
    if (next > stop_line_)
    { next = stop_line_; } }
```

신호에 따라 속도를 정하고 다음 위치를 계산하는 부분. 노랑이면 절반 속도로 가고, 빨강이면 정지선을 넘지 않도록 막는다. 이미 정지선을 지난 차는 빨강이어도 그대로 진행한다.

```
if (i > 0 && next > vehicles_[i - 1] - 40.0)
{
    next = vehicles_[i - 1] - 40.0;
}
if (next < vehicles_[i])
{
    next = vehicles_[i];
}
```

앞차와 최소 간격을 유지하는 부분. 벡터에 앞선 차부터 순서대로 들어 있어서 바로 앞 인덱스만 보면 된다. 아래쪽은 간격 때문에 위치가 뒤로 밀리는 것을 막는 처리다.

```
std::vector<float> alive;
for (size_t i = 0; i < vehicles_.size(); i++)
{
    if (vehicles_[i] < road_end_)
    {
        alive.push_back(vehicles_[i]);
    }
}
vehicles_ = alive;
```

화면 밖으로 나간 차를 지우는 부분. 남길 것만 새 벡터에 담아 통째로 교체했다.

```
light_sub_ = node->create_subscription<std_msgs::msg::String>(
    "traffic_light", 10,
    [this](const std_msgs::msg::String::SharedPtr msg)
    {
        ...
        Q_EMIT lightUpdated(QString::fromStdString(msg->data));
    });
```

Qt 쪽에서 두 토픽을 구독하고, 값이 오면 화면 쪽으로 신호를 보내는 부분. 통신 쪽 스레드에서 화면을 직접 건드릴 수 없기 때문에 신호로 넘긴다.

```
qRegisterMetaType<QVector<float>>("QVector<float>");
```

차량 위치는 숫자 하나가 아니라 배열이라 Qt가 기본으로 알지 못하는 타입이다. 신호로 스레드를 넘기려면 이렇게 미리 등록해줘야 한다.

```
void MainWindow::setLight(QString light)
{
    ui->redLight->setStyleSheet("background-color: rgb(70,70,70); border-radius: 20px;");
    ...
    if (light == "RED")
    {
        ui->redLight->setStyleSheet("background-color: rgb(255,50,50); border-radius: 20px;");
    }
    ...
}
```

신호등을 그리는 부분. 세 개를 먼저 전부 꺼진 색으로 만든 다음 해당하는 것 하나만 켜진 색으로 바꾼다. 둥근 모서리 값을 줘서 원형으로 보이게 했다.

```
void MainWindow::setVehicles(QVector<float> vehicles)
```

차량을 그리는 부분. 화면에 라벨 다섯 개를 미리 만들어두고, 받은 개수만큼만 위치를 옮겨 보여주고 나머지는 숨긴다. 매번 새로 만들지 않고 있는 것을 옮기는 방식이라 간단하다.

```
traffic_light_node:
  ros__parameters:
    r_time: 7.0
    g_time: 5.0
```

y_time: 0.0

신호등 파라미터 파일. 맨 윗줄의 이름이 노드 이름과 같아야 값이 적용된다.

```
Node(  
    package='traffic_sim',  
    executable='traffic_light_node',  
    name='traffic_light_node',  
    parameters=[light_config],  
    output='screen'  
),
```

launch 파일에서 노드 하나를 실행하도록 적는 부분. 세 개를 이렇게 나열해두면 명령 하나로 전부 실행된다. yaml 경로는 설치된 위치에서 찾아오도록 했다.

3. 파일구조

```
traffic_sim  
├── include  
│   ├── traffic_sim  
│   ├── main_window.hpp  
│   ├── qnode.hpp  
│   ├── traffic_light_node.hpp  
│   └── vehicle_node.hpp  
├── src  
│   ├── main.cpp  
│   ├── main_window.cpp  
│   └── qnode.cpp  
├── nodes  
│   ├── traffic_light_node.cpp  
│   └── vehicle_node.cpp  
├── ui  
│   └── mainwindow.ui  
├── config  
│   ├── traffic_light.yaml  
│   └── vehicle.yaml  
└── launch
```

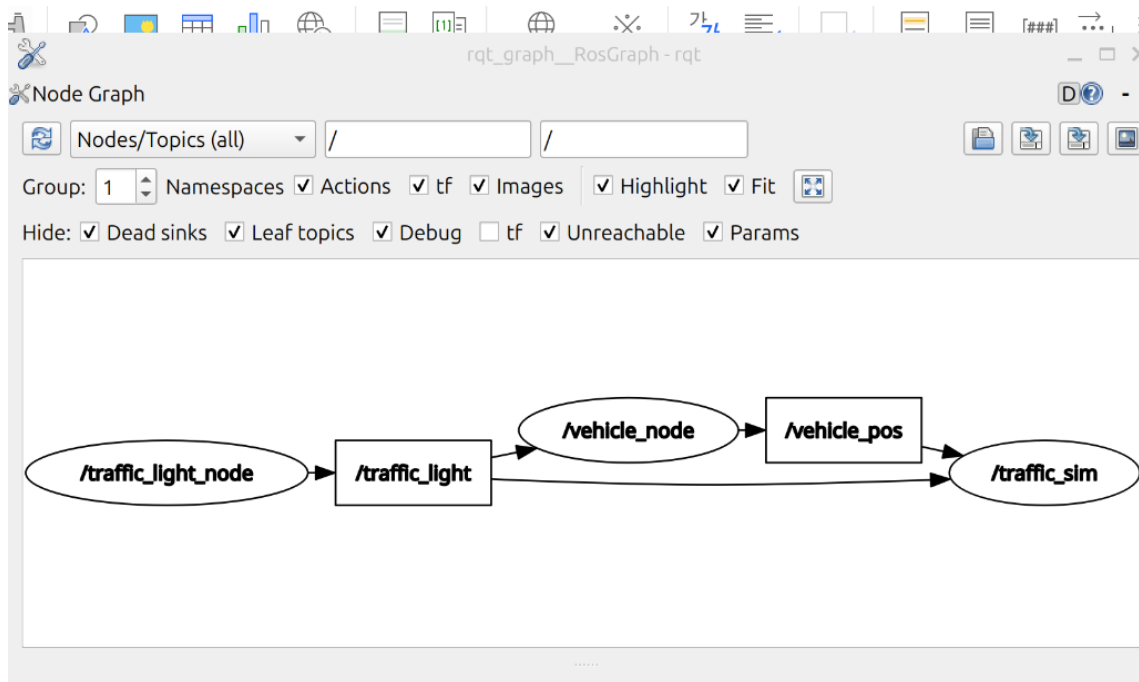
```

|   └── traffic.launch.py
|   └── resource
|   └── CMakeLists.txt
|   └── package.xml

```

실행은 `ros2 launch traffic_sim traffic.launch.py` 한 줄이면 된다.

4. 실행결과



트레픽 라이트 노드에서 오는 트레픽 라이트 토픽은 두 갈래로 나뉜다.

아래쪽 갈래 부터 보자면 라이트 토픽이 트레픽 심으로 가서 신호등의 색깔을 정하게 된다.

위쪽 갈래로 뻗어나간 토픽은 차량 노드를 통해서 차량의 위치를 계산하고 이 위치 토픽을 트레픽 심으로 보내서 차량을 나타낸다.

*첨부 영상 참고